

MCU

Theoretical background

Microcontrollers (MCU) are a fundamental element of modern embedded systems, found in consumer electronics, automotive, industrial automation and IoT devices.

Goal

Your task is to create an MCU (Micro Controller Unit) with the given set of instruction and memories for program and data. You get from the Judge a block of data (LSB first) that you must store in the Program Memory and Data Memory. Then you must run execution from address 0x000. When you reach STOP (or illegal) instruction then send back a content of the Data Memory (1024B) to the Judge.

The processor has:

- 16-bit instruction word
- Two 8-bit general purpose register: R0, R1
- Carry flag

Table 1 Memory Map

0 1023	Program Memory (16bits 512 Words)	
1024 2047	Data Memory (8bits 1024 Bytes)	

Table 2 Instruction Set

Mnemonic	Binary Code	Description
NOP	0000000000000000	No operation
STOP	0000100000000000	Stop execution and send back Data Memory
CLRC	0001000000000000	Clear Carry flag
SETC	0001100000000000	Set Carry flag
JMP	00100000aaaaaaaa	Jump
CJMPNC	00101000aaaaaaaa	Conditional jump if Not Carry
CJMPC	00110000aaaaaaaa	Conditional jump if Carry set
LDRM Rx aa	01000xaaaaaaaa	Load register Rx from Data Memory $Rx \leftarrow DM[aa]$
STRM Rx aa	01001xaaaaaaaa	Store register Rx to Data Memory $DM[aa] \leftarrow Rx$
LDRI Rx ii	01010x00iiiiiiii	Load register Rx with immediate Operand $Rx \leftarrow ii$
ADDR Rx Ry	01011xy000000000	Add registers with Carry, set Carry bit if carry $\{C, Rx\} \leftarrow Rx + Ry + C$
SUBR Rx Ry	01100xy000000000	Subtract registers with Carry $\{C, Rx\} \leftarrow Rx - Ry - C$
ROL Rx	01101x0000000000	Rotate register left $\{C, Rx\} \leftarrow (Rx \ll 1) \mid Rx[7]$
ROR Rx	01110x0000000000	Rotate register right $\{Rx, C\} \leftarrow (Rx \gg 1) \mid (Rx[0] \ll 7)$
SHL Rx	10000x0000000000	Shift register left $\{C, Rx\} \leftarrow (Rx \ll 1)$
SHR Rx	01111x0000000000	Shift register right $\{Rx, C\} \leftarrow (Rx \gg 1)$
ASHR Rx	10001x0000000000	Arithmetic shifts register right $\{Rx, C\} \leftarrow (Rx \gg 1) \mid (Rx[7] \ll 7)$
NAND Rx Ry	10010xy000000000	Bitwise NAND $Rx \leftarrow Rx \text{ nand } Ry$
XOR Rx Ry	10011xy000000000	Bitwise XOR $Rx \leftarrow Rx \text{ xor } Ry$

Any other opcode should be treated as a STOP command.

Provided module

You can edit only the *task_14.sv* file. If you wish, you may add other files for supportive submodules.

The module has the following inputs and outputs:

Table 3 List of inputs and outputs of the task_14 module.

Signal name	Signal type	Bit length
i_clk	Input	1 bit
i_valid	Input	1 bit
i_rst	Input	1 bit
i_data	Input	8 bits
i_last	Input	1 bit
i_first	Input	1 bit
o_data	Output	8 bits
o_valid	Output	1 bit
o_last	Output	1 bit

The reset is synchronous and active high.

Input and output interfaces

In one packet, the task receives a sequence of data on the *i_data* port. The maximal amount of data is 2KB. The beginning of the data block is marked by the *i_first* signal and the end by the *i_last* signal. Throughout the data transfer, the *i_valid* signal is high.

Task receives data in the way shown in the figure below:

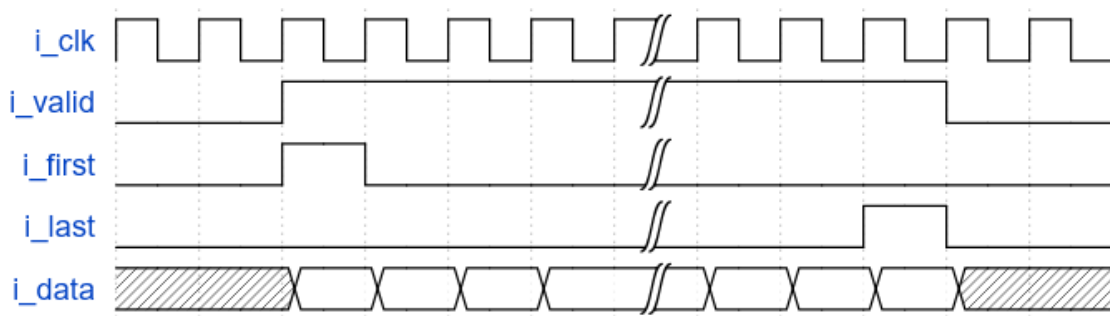


Figure 1 Input waveforms.

To send data from the task in a proper way one needs to provide the *o_valid* signal that is synchronized with data on the *o_data* bus and *o_last* signal that is synchronized with the last sample in output packet.

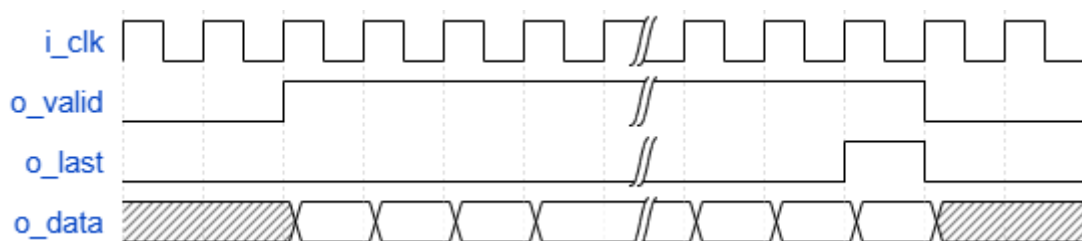


Figure 2 Output waveforms.

Output data can be sent as a continuous stream, or it can have regular or irregular gaps. The only requirement is providing *o_valid* and *o_last* signals.

Evaluating the task

The evaluation is done by comparing the sequence returned by the task with the expected sequence calculated by the reference model in the Judge. If the sequences are the same, the task is marked as correctly solved.

In Development mode, the task will be tested using a single set of data that is randomized on each run. In Evaluation mode, the task will be tested using 3 fixed sets of data, which remain the same across all runs and are identical for every team (for more information, check chapter 2.3.1 **Testing modes** in the **FPGA_Hackathon_2026_Project_Guide** document).

You earn 3 points for correctly processing each test vector. In the Development mode, this value simply indicates that the task passed the test. In the Evaluation mode, your total base score is calculated by multiplying the number of points by the number of test vectors (5), so you can earn up to 15 points.

Additional points (up to 5) may be awarded for **resource utilization when latency is below 15 μ s (1420 cycles)** (for more information, check chapter 2.3.2 **Bonus Points** in the **FPGA_Hackathon_2026_Project_Guide** document).